



0MÓDULO 4 · T12

Animaciones con Animated API en React Native

✨ *Animated API – flip de tarjeta Pokémon*

Carlos Rodríguez Contreras · GitHub: [karrocon](#)

¿Qué aprenderás hoy?

En esta sesión implementarás un efecto flip 3D sobre una tarjeta Pokémon usando la Animated API de React Native. Al terminar, comprenderás cómo controlar animaciones nativas de alto rendimiento y cómo interpolar valores para transformaciones visuales complejas.

1

Animated.Value y Animated.timing

Crear y controlar valores animados para disparar animaciones suaves y predecibles.

2

Interpolación de valores

Usar `interpolate` para mapear un único valor animado a grados de rotación en cada cara.

3

Efecto flip 3D

Implementar el giro completo de la tarjeta Pokémon con `rotateY` y `backfaceVisibility: 'hidden'`.

4

useNativeDriver: true

Entender por qué enviar la animación al hilo nativo garantiza 60 fps sin bloquear el hilo JS.

Animated API — conceptos clave

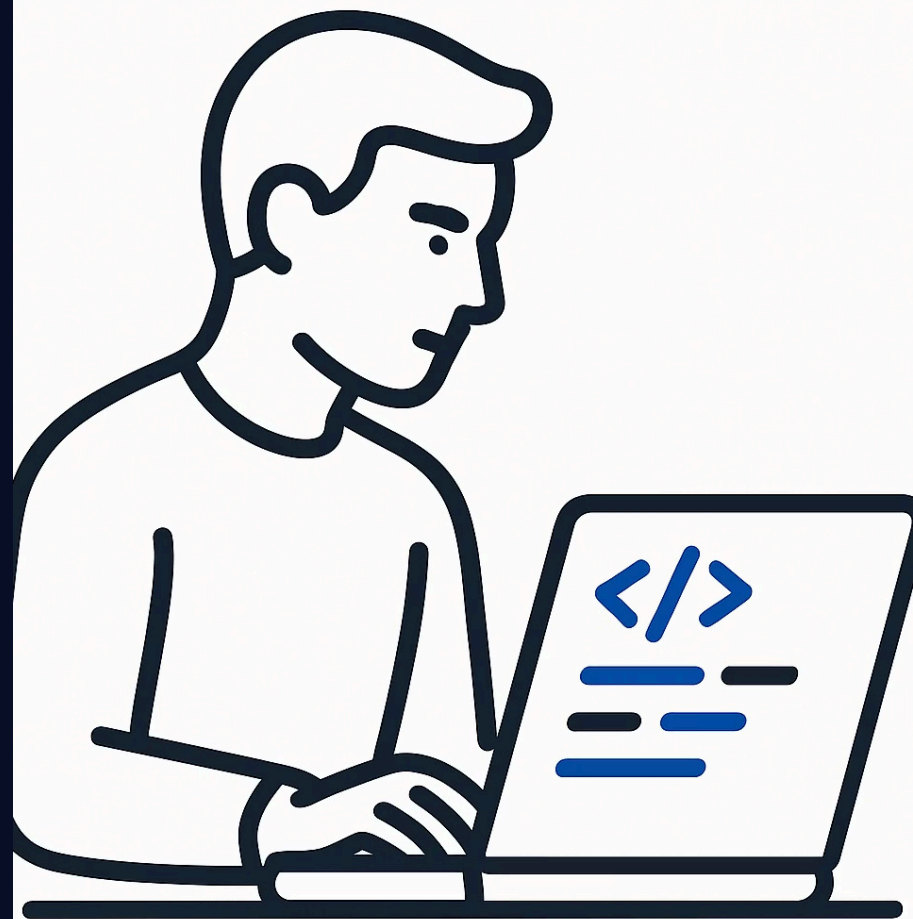
Toda animación con la Animated API sigue tres pasos: crear el valor animado, lanzar la animación y consumir ese valor en los estilos. Esta separación mantiene el código limpio y predecible.

```
// 1. Valor animado
const flipAnim = useRef(new Animated.Value(0)).current;

// 2. Animar
Animated.timing(flipAnim, {
  toValue: 180,
  duration: 300,
  useNativeDriver: true,
}).start();

// 3. Usar en estilos
<Animated.View style={{ transform: [{ rotateY: flipAnim }] }}>
```

- ❗ El valor numérico de `flipAnim` es animado por el sistema – tú solo defines el origen y el destino, React Native calcula los fotogramas intermedios.



useNativeDriver: true

¿Por qué usarlo siempre?

Por defecto, las animaciones se calculan en el hilo de JavaScript. Esto puede causar caídas de fotogramas cuando el hilo JS está ocupado renderizando o gestionando eventos.

Con `useNativeDriver: true` la animación se serializa y envía al hilo nativo antes de comenzar – el hilo JS ya no interviene durante la ejecución.

✔️ ☒ Sin bloquear el hilo JS · ☒ 60 fps garantizados

Limitación importante

```
Animated.timing(flipAnim, {  
  toValue: 180,  
  duration: 300,  
  useNativeDriver: true, //
```

Interpolación de valores

La función `interpolate` permite derivar múltiples valores a partir de un único `Animated.Value`. Para el flip de tarjeta, un solo `flipAnim` de 0 a 180 alimenta dos interpolados que calculan los grados exactos para cada cara.

```
const frontInterpolate = flipAnim.interpolate({
  inputRange: [0, 180],
  outputRange: ['0deg', '180deg'],
});
```

```
const backInterpolate = flipAnim.interpolate({
  inputRange: [0, 180],
  outputRange: ['180deg', '360deg'],
});
```

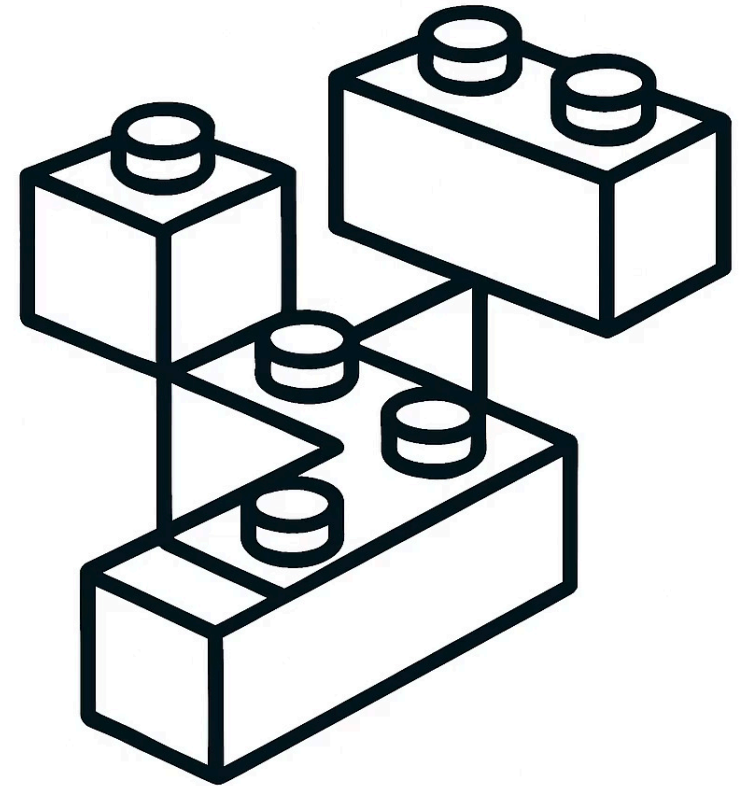
❗ El valor `flipAnim` va de 0 a 180, y los interpolados calculan los grados exactos para cada cara. La cara trasera empieza ya girada 180° y termina en 360°, completando el ciclo.

backfaceVisibility: 'hidden'

Para que el flip parezca real es necesario ocultar cada cara cuando está mirando hacia atrás. La propiedad `backfaceVisibility: 'hidden'` hace exactamente eso, y la cara trasera debe posicionarse en `absolute` para superponerse a la frontal.

```
// Cara frontal: visible de 0° a 90°, luego desaparece
<Animated.View style={{
  transform: [{ rotateY: frontInterpolate }],
  backfaceVisibility: 'hidden',
}}>
  {/* Frente de la tarjeta */}
</Animated.View>

// Cara trasera: posición absoluta, gira de 180° a 360°
<Animated.View style={{
  position: 'absolute', top: 0, left: 0,
  transform: [{ rotateY: backInterpolate }],
  backfaceVisibility: 'hidden',
}}>
  {/* Dorso de la tarjeta */}
</Animated.View>
```



PokeApp — DetallePokemon.tsx

Este es el componente completo tal y como aparece en la aplicación de ejemplo. Observa cómo `isFlippedRef` rastrea el estado del giro sin provocar rerenders, y cómo `stopAnimation()` cancela cualquier animación en curso antes de lanzar la siguiente.

```
export default function DetallePokemon({ pokemon }) {
  const flipAnim = useRef(new Animated.Value(0)).current;
  const isFlippedRef = useRef(false);

  const handleFlip = () => {
    flipAnim.stopAnimation(); // cancela animaciones en curso
    isFlippedRef.current = !isFlippedRef.current;
    Animated.timing(flipAnim, {
      toValue: isFlippedRef.current ? 180 : 0,
      duration: 220,
      useNativeDriver: true,
    }).start();
  };

  return (
    <TouchableWithoutFeedback onPress={handleFlip}>
      <Animated.View style={{ position: 'relative' }}>
        <Animated.View style={{
          transform: [{ rotateY: frontInterpolate }],
          backfaceVisibility: 'hidden'
        }}>
          <PokemonCardFront pokemon={pokemon} />
        </Animated.View>
        <Animated.View style={{
          position: 'absolute', top: 0, left: 0,
          transform: [{ rotateY: backInterpolate }],
          backfaceVisibility: 'hidden'
        }}>
          <PokemonCardBack pokemon={pokemon} />
        </Animated.View>
      </Animated.View>
    </TouchableWithoutFeedback>
  );
}
```

📌 **Buena práctica:** usa `useRef` para el estado del flip en lugar de `useState` – así evitas rerenders innecesarios que podrían interrumpir la animación.

Tour starter/FlipCard.tsx

Este es el fichero que encontrarás en la carpeta `starter/`. Contiene la estructura básica con los puntos marcados como `TODO` que deberás completar durante el ejercicio. El componente ya incluye el `TouchableWithoutFeedback` y el `View` contenedor – tu misión es añadir toda la lógica de animación.

```
export default function FlipCard() {  
  // TODO: const flipAnim = useRef(new Animated.Value(0)).current;  
  
  const handleFlip = () => {  
    // TODO: anima el giro  
  };  
  
  return (  
    <TouchableWithoutFeedback onPress={handleFlip}>  
      <View style={styles.container}>  
        {/* TODO: Animated.View cara frontal */}  
        {/* TODO: Animated.View cara trasera (position absolute) */}  
      </View>  
    </TouchableWithoutFeedback>  
  );  
}
```

⚠ No modifiques los estilos base del `StyleSheet` – céntrate en los `TODO` de la lógica de animación e interpolación marcados arriba.

Ejercicio T12

Pon en práctica todo lo aprendido. Abre el proyecto en tu editor y sigue los pasos en orden – cada uno construye sobre el anterior. Implementarás el flip completo desde cero partiendo del fichero `starter/FlipCard.tsx`.

1 Abre el fichero de inicio

Navega a `starter/FlipCard.tsx` en tu editor.

2 Añade `flipAnim`

Crea el valor animado con `useRef(new Animated.Value(0))`.

3 Implementa `handleFlip`

Añade `Animated.timing` con `useNativeDriver: true` y alterna entre `toValue: 0` y `180`.

4 Crea los interpolados

Implementa `frontInterpolate` y `backInterpolate` con sus rangos de entrada y salida correctos.

5 Aplica `backfaceVisibility` y posición absoluta

Configura `backfaceVisibility: 'hidden'` en ambas caras y `position: 'absolute'` en el dorso.

Resumen de la sesión

Estos son los conceptos clave que has trabajado hoy. Dominarlos te permitirá implementar animaciones fluidas y de alto rendimiento en cualquier componente React Native.

Concepto	Sintaxis clave	Para qué sirve
Animated.Value(0)	<code>new Animated.Value(0)</code>	Valor numérico animable
useRef	<code>useRef(...).current</code>	Persiste el valor entre renders
Animated.timing	<code>Animated.timing(val, { toValue, duration })</code>	Anima a un toValue en duration ms
interpolate	<code>val.interpolate({ inputRange, outputRange })</code>	Mapea rangos de valores
useNativeDriver	<code>useNativeDriver: true</code>	Animación en hilo nativo (60 fps)
backfaceVisibility	<code>backfaceVisibility: 'hidden'</code>	Oculto la cara trasera
position: 'absolute'	<code>position: 'absolute', top: 0, left: 0</code>	Superpone las dos caras

✅ ¡Enhorabuena! Ahora tienes las herramientas para crear animaciones nativas fluidas y eficientes en cualquier proyecto React Native. 🚀